Note

The dialogs, which the functions open are modal. The simulation only continues when the user reacted to the requested interaction.

prompt [SimTalk]

Asks the user to enter data into a dialog.

Remarks

The dialog, which the function opens is modal. The simulation only continues when the user reacted to the requested interaction by clicking the **OK** button in the dialog.

Type

Function

Syntax

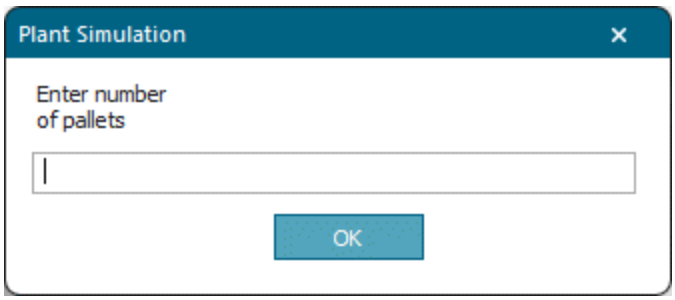
```
prompt(Text:string[, MoreText:string, ...]) → string
```

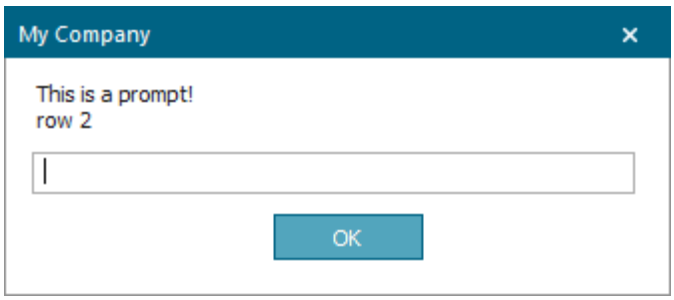
Parameters

You can specify the following parameters:

- The parameter *Text* of data type *string* designates the text you want to show. *Plant Simulation* opens a dialog displaying the designated text and a text box. It pauses the simulation run until the user has entered that data and closed the window by clicking **OK**. *Plant Simulation* can display a maximum of four lines with a maximum of 45 characters per line. It treats each line as a parameter of data type *string* of its own.

  Type in the text, which you want to show in the title bar of the dialog, between two vertical bars, for example `("|My Company|This is a prompt!", "row 2")`.
- The optional parameter *More Text* of data type *string* designates additional text that you want to show.





Return Value

The return value has the data type *string*.

Is the data the user entered.

Note

You can convert the result with the functions for converting data types.

Examples

```
// method for querying the number of pallets in a model
-> integer
var s: string
// gets the number of pallets
s := prompt("Enter number", "of pallets")
return str_to_num(s)
```

```
var s: string
s := prompt("|My Company|This is a prompt!", "row 2")
```

See also

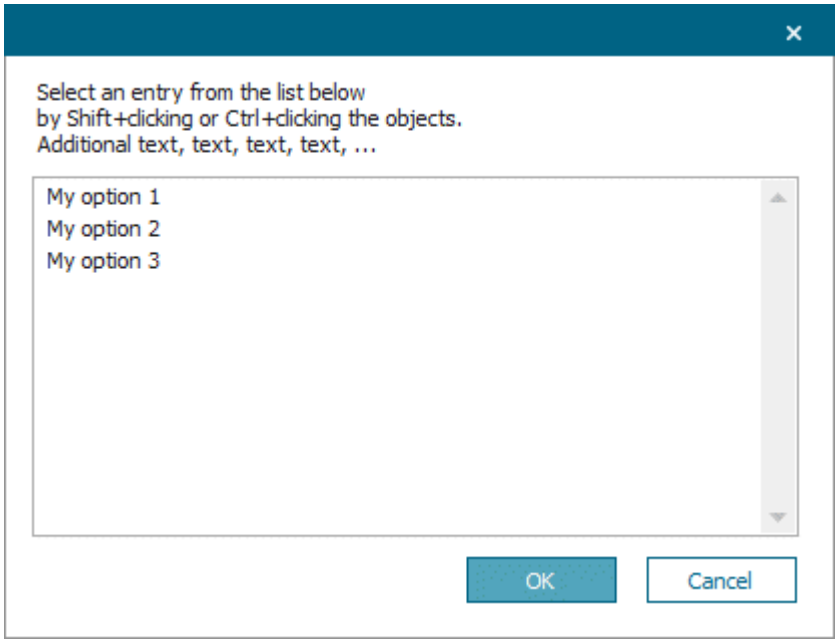
Functions for Converting Data Types

promptList1 [SimTalk]

Enables the user to pick a **a single** entry from the list, which is displayed in the message window.

Remarks

The user can select **a single** entry and click **OK**.



The dialog, which the function opens is modal. The simulation only continues when the user reacted to the requested interaction by clicking the respective button in the dialog.

Type

Function

Syntax

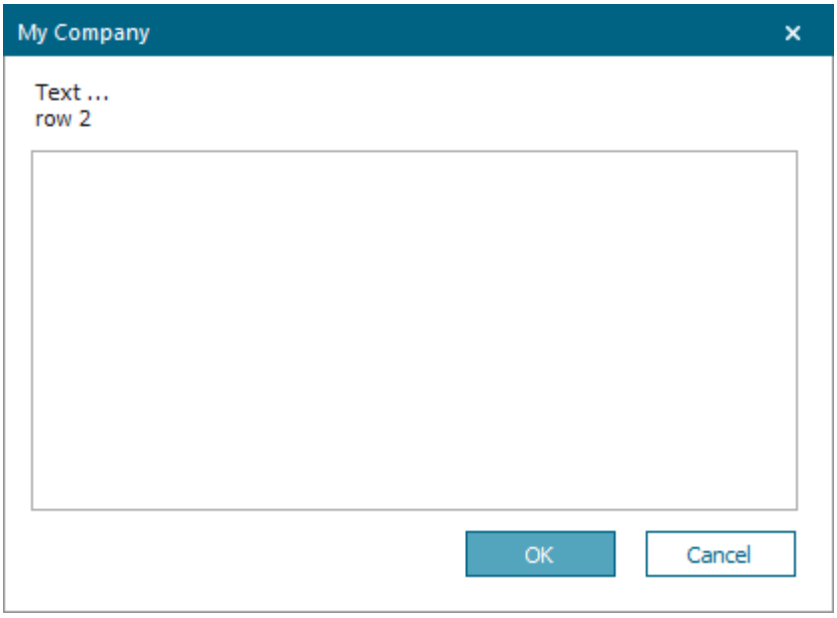
```
promptList1(Entry:list[, Text:string, ...]) → integer
```

Parameters

You can specify the following parameters:

- The parameter *Entry* of data type *list* designates a list of any type with one column.
- The optional parameter(s) *Text* of data type *string* designate the text above the list box. You can enter any number of optional parameters of data type *string* with a maximum length of 45 characters.

Type in the text, which you want to show in the title bar of the dialog, between two vertical bars, for example `(DataList, "|My Company|Text ...", "row 2")`.



Return Value

The return value has the data type *integer*.

Is the index of the selected entry.

0 when the user clicks **Cancel**.

Examples

```
promptList1(DataList, "Select an entry from the list below",
"additional text, text, text, text, ...")
// the entries are contained in the object DataList
```

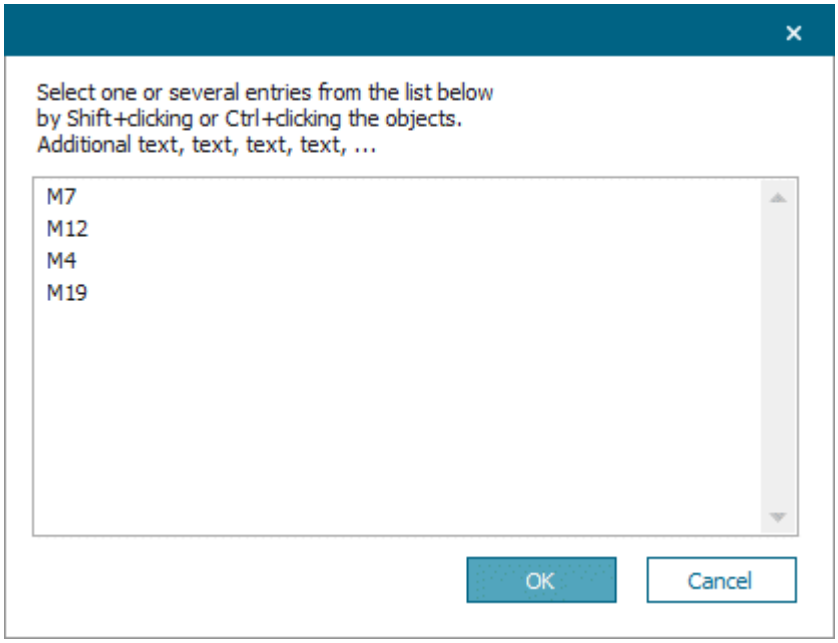
```
promptlist1(DataList, "|My Company|Text ...", "row 2")
```

promptListN [SimTalk]

Enables the user to pick **one or several entries** from the designated list, which is displayed in the message window.

Remarks

The dialog, which the function opens is modal. The simulation only continues when the user reacted to the requested interaction by clicking the respective button in the dialog.



The user can select **one or several entries** by **Shift/Ctrl+clicking the objects** and click **OK**.

Type

Function

Syntax

```
promptListN(List:list[, Text:string, ...]) → list
```

Parameters

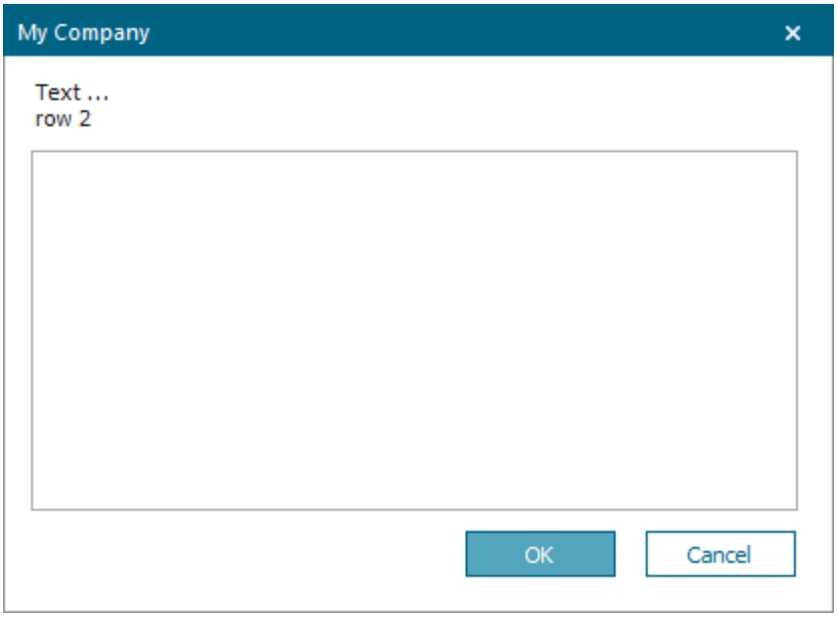
You can specify the following parameters:

- The parameter *List* of data type *list* designates the list.

The user can select **one** or **more** entries by holding down **Ctrl** and clicking the entry/entries. **OK** closes the dialog and accepts the choice.

- The optional parameter(s) *Text* of data type *string* designate the text above the list box. You can enter any number of optional parameters of data type *string* with a maximum length of 45 characters.

  Type in the text, which you want to show in the title bar of the dialog, between two vertical bars, for example `(DataList, "|My Company|Text...", "row 2")`.



Return Value

The return value has the data type *list[integer]*.

When the user clicks **Cancel**, *Plant Simulation* returns 0.

Plant Simulation saves the indexes of the selected entries unsorted.

Examples

```
// A part entering the plant may be processed by more than one machine.
var machPark: list[string]  // You are going to enter the machines that can
var indices: list[integer]  // process it
machPark.create
machPark[1] := "M7"; machPark[2] := "M12"
machPark[3] := "M4"; machPark[4] := "M19"
indices := promptListN(machPark,
   "Select one or several of the machines that can", "process the part:")
for var i := 1 to Indices.Dim
   switch indices[i]
   case 1
   // M7 selected
   case 2
```

```
      // M12 selected
   case 3
      // M4 selected
   case 4
      // M19 selected
   end // switch
next
```

```
promptlistN(MyDataList, "Select one or several entries from the list below",
"by Shift+clicking or Ctrl+clicking the objects.","Additional text, text,
text, text, ...")
// the entries are contained in the object MyDataList
```

```
promptlistn(DataList, "|My Company|Text...", "row 2")
```

Output Functions

Output Functions

SimTalk provides the functions listed in the table of contents to the left for outputting data.

beep [SimTalk]

Plays a beeping sound on the computer speaker. You can use it to alert the user about messages or critical situations.

Type

Function

Syntax

```
beep
```

Example

```
if store.full
   beep // acoustic alert
   print "The Store is full"
else
   @.move(Store)
end
```

bell [SimTalk]

Outputs an acoustic signal on the computer speaker.

Type

Function

Syntax

```
bell(Frequency:integer, Duration:integer)
```

Parameters

You can specify the following parameters:

- The parameter *Frequency* of data type *integer* designates the frequency of the bell sound.
- The parameter *Duration* of data type *integer* designates its duration.

Example

```
bell(440,1000)
```

getUnit [SimTalk]

Returns the unit of the passed value of the data type *length*, *weight*, *time*, *speed*, *money*, or *acceleration*.

Remarks

This is the unit which you selected under **File > Model Settings > Units**.

Type

Function

Syntax

```
getUnit(Value:any) → string
```

Parameter

The parameter *Value* of data type *any* designates the value for which you would like to get the unit.

Return Value

The return value has the data type *string*.

Example

```
var l: length := 100 // meters

// We now assume that the length unit in the Model Settings is set to
kilometers
print l          // returns 0.1 km
print to_str(l)  // returns 0.1, to_str does not add a unit
print getUnit(l) // returns km

// Since we don't have settings for areas we still get SI units here
print l*l          // returns 10000m²
print getUnit(l*l) // returns m²
```

See also

Units [model settings]

infoBox [SimTalk]

Presents the user with a message box, which shows the designated text.

Remarks

Close the message box by calling the function *infoBox* a second time and by passing an empty string `""`.

> **Note**
> Open a modal *Infobox* to prevent the user from modifying a simulation model during a simulation run.

If the *Debugger* is opened, any open message box will be closed. Otherwise, an open modal message box would prevent you from continuing your work.

> **Note**
> If you inadvertently opened a modal message box without closing it, hold down the **Shift+Ctrl+Alt keys** for 5 seconds. This closes the message box and opens a dialog you can close by clicking **No**.

Type

Function

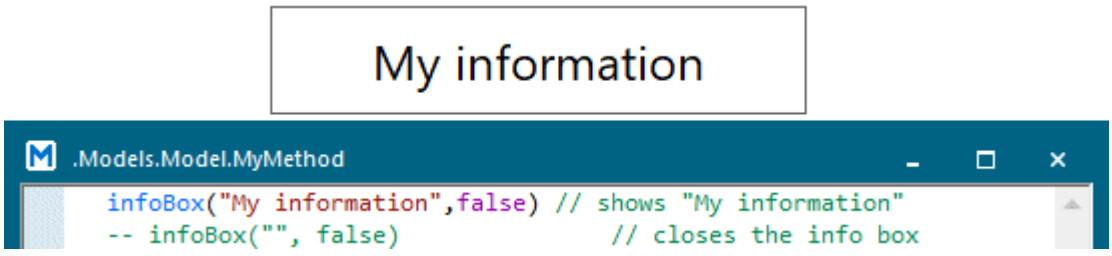
Syntax

```
infoBox(Text:string, Modal:boolean)
```

Parameters

You can specify the following parameters:

- The parameter *Text* of data type *string* designates the text, which the message box shows.
- The parameter *Modal* of data type *boolean* sets if the dialog is modal, i.e., that you cannot access any other *Plant Simulation* window (`true`) or not (`false`).



Example

```
infoBox("My information",false)  // shows "My information"
infoBox("", false)               // closes the info box
```

print [SimTalk] - output function

Outputs any amount of text to the *Console*.

Remarks

You can open the *Console* by clicking the  button on the **Window** ribbon tab.

Strings within *arrays* are enclosed in quotation marks.

Type

Function

Syntax

```
print ...
```

Parameter

For values of data type *length*, *weight*, *speed*, and *acceleration*, the *Console* shows the unit after the value.

You can specify any number of parameters without any type constraint. You can use this function to show error messages or other hints for the user.

Examples

```
print "Machine: ",   station.name,  "  - is blocked"
```

```
@.Cont.move(MyDrain)
print @, ":"
print "Previous destination: ", @.Destination  // writes the previous
                                               // destination to the Console
@.Destination := LoadingStation
print "New Destination: ", @.Destination      // writes the new
                                              // destination to the Console
print                                         // inserts an empty line
```

```
var a : string["Kent, Clark", ""]
var s : string := to_str(a)  -- assigns ["Kent, Clark", ""]
```